

## Linux基础

- 简介linux
- 简介linux发行版
- linux特点
- 分区
- 分区文件系统
- 启动流程
- linux下的引导程序
- windows配置不考
- aptget原理基本不考，yum也不
- 虚拟终端
- 命令
- 文件类型
- 文件夹结构
- 文件权限
- 进程的概念
- 帮助文档
- 文件命令
- 重定向考试重点！
- 管道
- 环境变量
- 高级命令与正则表达式

## shell

- shell进程有什么特点，是什么程序
- shell在层次的位置
- shell只考bash，但应该知道其他
- shell扮演的角色，哪些功能是shell执行的？管道？ls？重定向？哪些是shell执行哪些不是？
- 执行shell的三个方法及其区别
- shell别名，输出环境变量
- 默认执行脚本
- 即时文档要考

## 编译

- linux可执行码是elf格式
- 编译链接图
- 静态库和动态库区别
- Delphi不考，java&.Net要能说出来
- 编译器结构

- GCC

- 调试器(GDB)

- MakeFile

  - make

  - makefile语法

  - 伪目标

  - makefile例子

  - 预定义变量

## 系统调用

- 文件、文件系统概念，考主观题
- 文件类型！

VFS重点

#### 4.3.1. 什么是VFS

文件系统在磁盘上的格式完全不考

硬链接和软连接的区别!

系统调用和库函数有什么联系和区别

考试编程

## 内核

内核概念

内核层次结构知道就行

源码获取方式、编译步骤不考

启动内核的配置、引导要掌握 (kernel initrd initrd) , kernel initrd initrd怎么获得不考

打包不考

驱动命令都要

模块间依赖知道是怎么回事

内核代码、用户态代码的联系和区别

写内核的注意事项

helloworld内核代码尽量不考, 考概念为主

怎么编译内核模块不考

跟硬件打交道不考

传参不考

内核间的依赖, 知道导出的符号有哪几种, 查看导出符号的命令要掌握

proc文件是个什么东西, 主观题, 相关函数不考

驱动有哪几种类型

建立字符型设备的具体步骤, 主观题

整个接口原理、驱动建立之后用户态程序如何使用

设备号

华为讲课内容

# Linux基础

## 简介linux

基本不考, 考就随便说说, 开源、好用、很多发行版本之类的。linux历史不考

### 1. Linux 是什么样的系统

1. Linux是使用C编译器、C依赖库、emacs和bash开发的。
2. 根据GNU通用公共许可证开发的免费Unix类型的操作系统。
  - 开源
  - 受欢迎
  - 支持大多数可用平台

### 3. GPL协议

### 2. Linux 创始人: Linus Torvalds

### 3. Linux 发行版: Red Hat(后来发展为Fedora)、Debian(一定要配置自由软件, 不会配置商业软件)、Ubuntu(用户好用就行)、SuSe、Mandrake、Redflag

# 简介linux发行版

---

挑两个写，写点特点就行了。

1. redhat: 全称Redhat Enterprise Linux, 顾名思义, 这是为企业打造的Linux系统, 特点是**稳定并且收费**, 非常适合于**服务器使用**, 一般**大型企业**的重要服务器都是用的rhel系统, rhel大概是最经典的Linux发行版, 在很多人看来, rhel就等同于Linux, 至于redhat公司的红帽认证, 更是运维人员含金量最高的认证证书。软件包管理器为rpm, 软件包管理器的前端工具为yum。
2. Ubuntu是国内很多初学者第一次接触到的Linux发行版, 其桌面版系统受到众多Linux爱好者推荐。她基于Debian的unstable版本加强而来, 可以这么说, Ubuntu就是一个拥有Debian所有的优点, 以及自己所加强的优点的近乎完美的Linux桌面系统。根据选择的桌面系统不同, 有三个版本可供选择, 基于Gnome的Ubuntu, 基于KDE的Kubuntu以及基于Xfc的Xubuntu。特点是**界面非常友好, 容易上手**, 对**硬件**的支持非常全面, 是非常适合做桌面系统的Linux发行版本。

## linux特点

---

linux有什么特点、有什么优势、为什么要学linux。linux协议不考

1. 大多数软件(包括Linux内核)均为GPL版(GNU通用公共许可证)
2. 称为"copyleft"(而不是"copyright")
  1. 您可以复制软件
  2. 获得源代码
  3. 您可以更改源代码并重新编译
  4. 您可以分发更改后的源和二进制文件
  5. 您可能会为此付费
  6. 您只能更改许可证
3. 因此, 所有客户都享有与您相同的权利
4. 因此, 您真的无法通过出售单独的软件。
5. 其他开源许可证(例如BSD)也可用

## 分区

---

知道磁盘有MBR GPT主要概念知道一下

1. 分区的基本知识
  1. 最多只能有四个主分区; 主分区可以是扩展分区; 一个扩展分区可能有很多逻辑分区。
2. MBR硬盘分区
  1. 磁盘分区 不能超过4T
  2. 扩展磁盘分区 扩展分区不能直接使用, 还需要划分为逻辑分区, 额外分区是指向正确的额外分区表
  3. 逻辑分区
2. MBR: Master Boot Record: 主引导记录
  1. 磁盘的0磁道的第1个扇区称为MBR, 共512字节, 由BIOS定位
  2. 内容
    1. 第一部分有446个字节, 用于存放引导代码, 即是bootloader。
    2. 第二部分有64个字节, 用于存放**磁盘分区表**。其中, 每个分区的信息需要用**16个字节**来记录。因

此，一个硬盘最多可以有4个分区，这4个分区称之为主分区和扩展分区(extended)。

3. 2 bytes的magic number"幻数"(0x55AA)

3. GPT(GUID Partition Table) Globally Unique Identifier Partition Table

1. 新的硬件必须有新的代码与其对应
2. 不要在老的机器上装新的分区组织方式
3. GPT是MBR不够用了才会选择使用
4. 支持上百个主分区，不需要扩展分区

## 分区文件系统

---

- 操作系统中负责存取和管理文件的部分。
- 文件及其某些属性的集合。它为引用这些文件的文件序列号提供名称空间。
- 类型
  - VFS, (Virtual File System) 虚拟文件系统, 与以下的磁盘文件系统(即文件的分区格式不同), 为底层的文件系统提供了统一的抽象
  - EXT2, EXT3, EXT4, FAT32, ExFAT

## 启动流程

---

不会考的很细

重点bootloader和内核

1. 打开电源 -> BIOS -> boot loader -> Linux kernel -> init -> System Ready
2. BIOS (基本的输入输出系统): 检查内存, 从非易失内存中加载参数(内存时序、启动设备顺序)、检查启动设备(软盘、CD-ROM、磁盘等)、装载启动设备的MBR(包含"Boot Loader"和分区表, 通常是有LILO/GRUB来启动)并执行
3. MBR:
  1. 包含一个"引导加载程序"和分区表
  2. 传统上由LILO/GRUB建立
4. Boot loader:
  1. 将压缩后的内核映像加载到Linux内核内存中
  2. 内核解压并启动
5. 初始化进程
  1. 使用/etc/inittab的配置文件
  2. 按照启动级别启动

## linux下的引导程序

---

知道grub、grub配置项要知道kernel与initrd

- root 告诉 GRUB OS 文件系统的实际位置。可见, GRUB 引用介质的方式与 LILO 不同。在 LILO 示例中, /dev/hdb3 是第二块硬盘中的第三个分区。Grub 将此硬盘引用为 (hd1,3), 即第二块硬盘的第三个分区 (disk 0 是第一块硬盘)。

- kernel: kernel 是Grub的一个命令, 用来加载Linux内核。名为“/boot/vmlinuz-XXX”的内核将被载入内存, 并读取“root=UUID=XXX”这个分区的一些必需的文件。(vmlinuz-X.X.XX-XX 是 root 目录中默认引导映像的名称。)

```
kernel /boot/vmlinuz-2.6.20-16-generic root=UUID=3f784cd9-516f-4808-a601-b19356f6bdea ro quiet splash locale=zh_CN vga=0x318
```

- initrd: 先用名称为“/boot/initrd.img-XXX”的文件, 在内存中构造一个“虚拟”的根文件系统, 以便于内核检测完硬件信息后载入必需的模块等等。(initrd-X.X.XX-XX.img 是 root 目录中默认 initrd 映像的名称。)

```
initrd /boot/initrd.img-2.6.20-16-generic
```

1. 引导加载程序加载并启动Linux内核: 向内核传递启动参数(设备信息)、选择加载初始根磁盘、启动其他操作系统。
2. 通用引导加载程序: LILO(Linux加载程序)、GRUB(Grand Unified Boot Loader)
3. 除非使用其他引导加载程序, 否则通常在/dev/hda中进行配置

## windows配置不考

---

## aptget原理基本不考, yum也不

---

## 虚拟终端

---

知道7个分别是什么

linux下的纯命令行界面, 一般会有6个虚拟终端, 其中第七个为图形化界面。ctrl+alt+fn切换

在大多数Linux发行版中, 控制台模拟许多虚拟终端。每个虚拟终端都可以看作是一个独立的、直接连接的控制台。不同的用户可以使用不同的虚拟终端

1. VT 1-6 文本模式登陆
2. VT 7 图形模式登陆提示
3. 可以使用Alt-Fn在VT之间切换

## 命令

---

除了特别说明不考的都考, cal、finger不考

PPT没提到但很常用的也会考

- passwd: 修改密码
- mkpasswd: 生成随机密码
- date: 当天日期
- cal: 展示一个日历
- who: 查看用户名以及当前登录名
- whoami: 查看当前用户名

- `finger`：当前用户的home目录
- `clear`：清空当前屏幕
- `echo`：写一个信息到屏幕；-n 不换行
- `write`：为已经登录的用户发送一个消息，给其他用户发信息
- `wall`：群发消息，write all给所有登录到系统的用户发信息
- `talk`：希望进行对象，建立聊天session
- `mesg`：用来设置是否允许向当前终端写消息(Y/N)，可以屏蔽用户发来的信息
- `pwd`：打印工作目录
- `cd`：切换文件夹
- `mkdir`：创建文件夹
  - `-m` 设置权限(-m 777)
  - `-p` 一次性创建多个路径
  - `-v` 每次创建新目录都显示信息
- `rmdir`：删除文件夹
- `ls`：查看信息：`-l` 查看所有信息；`-a` 查看包含隐藏文件；`-R` 递归的查看所有文件信息；格式为 文件类别、权限(3种)、链接数(硬链接数)、拥有者、拥有者组、大小、最后修改时间、名称
- `touch`：更新权限或修改文件权限
  - `-a` 更改File变量指定的文件的访问时间，不更新修改时间
  - `-c` 如果文件不存在，则不要进行创建
  - `-f` 强制touch运行，不需要管理文件的读和写许可权
  - `-m` 更改FILE的修改时间，不要更改访问时间。
- `cp`：拷贝文件
- `mv`：移动或重命名文件
- `ln`：链接文件
- `rm`：删除文件
- `cat`：打印文件内容
- `more`：显示文件内容，向下滚动
- `less`：显示文件内容，上下滚动
- `ps`：报告所有的进程状态
- `pstree`：展示树状的程序们
- `jobs`：用于显示Linux中的任务列表及任务状态，包括后台运行的任务；显示任务号及其对应的进程号。
- `fg`：将一个进程移动到前台执行
- `bg`：将一个进程移动到后台执行
- `<ctrl-z>`：停止某一个任务
- `kill`：杀死一个进程
- `nohup`：忽略挂起的信号

- `nice`：以更改过的优先级运行进程
- `renice`：重新指定一个或多个进程的优先级
- `top`：查看当前的CPU信息状态
- `mknod`：在/dev下创建驱动的设备文件
- `mkfifo`：创建管道文件
- `chmod`：修改文件权限
  - `who`：u, g, o, a
  - `Operator`：+, -, ==
  - `what`：r(4), w(2), x(1)
- `chown`：修改文件所有者
- `chgrp`：修改文件分组
- `find`：查找(find dir [-name] "\*.c")
  1. options
  2. -depth 在查看目录本身之前，先搜索目录的内容
  3. -follow 跟随符号链接
  4. -maxdepths N 最多搜索N层目录
  5. -mount/-xdev 不搜索其他文件系统中的目录
  6. tests
  7. -name 根据名字查找
  8. -type 根据文件类型查找
  9. -user 根据用户名查找
  10. -ctime 根据最近更新时间查询
  11. -atime N 文件在N天之前被最后访问过
  12. -mtime N 文件在N天之前被最后修改过
  13. -newer other\_file 文件比other\_file新
- `ar`：用于建立或修改备存文件(创建静态链接库)，或是从备存文件中抽取文件。
- `tar`：
  1. 解压：tar -xvzf test.tar.gz
  2. 压缩：tar -zcvf test.tar.gz ./test/
- `head`：查看文件头部，默认10行
- `tail`：查看文件尾部，默认10行
- `su`：切换用户名
- `uname`：打印当前Linux的版本信息
- `man`：查看命令说明书
- `grep`：在文件中搜索字符串

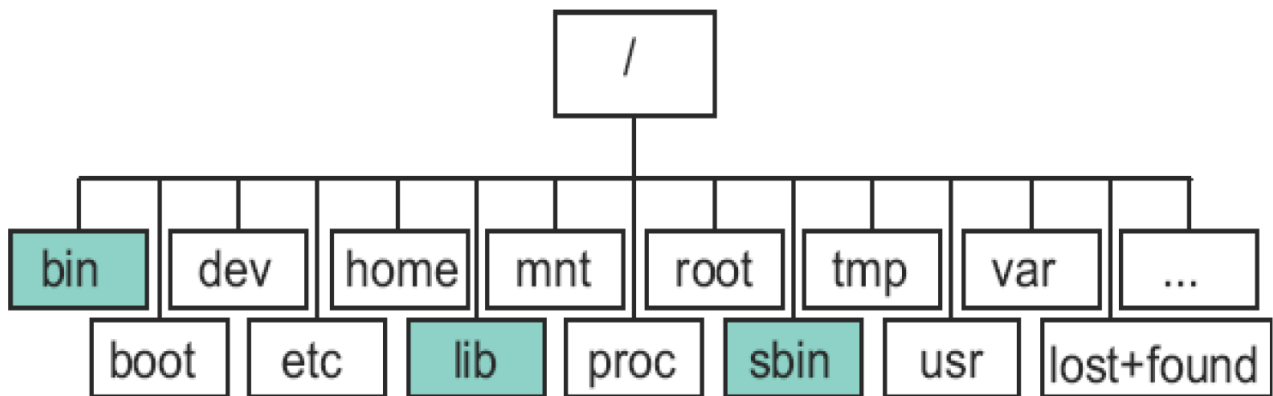
## 文件类型

---

文件概念不考，文件系统类型很大概率会考（七种）

## 文件夹结构

---



# Linux Filesystem Hierarchy Standard:

## <http://www.pathname.com/fhs>

主要的应该知道

1. `/boot`：内核、bootloader的配置，包括引导加载程序相关的文件
2. `/etc`：系统的配置文件所在地(包含下载的软件的配置)
3. `/bin`：程序文件夹，包含二进制可执行文件，例如 `ls`，其实是在执行一个程序，也有一部分程序在 `/usr/bin` (在我的linux上，`/bin`是`/usr/bin`的软链接)
4. `/sbin`：系统二进制文件，但是这个目录下的Linux命令通常由系统管理员使用，对系统进行维护，例如 `ifconfig/fdisk` 也有部分程序在 `/sbin`，例如分区命令 `fdisk`
5. `/mnt`：挂载目录，临时挂载目录，系统管理员可以挂载文件系统。
6. `/usr`：资源文件夹(和编程相关的)；编译器、默认的头文件、系统中的库文件，包含二进制文件、库文件、文档和二级程序的源代码
  1. `/usr/bin` 中包含用户程序的二进制文件。
  2. `/usr/sbin` 中包含系统管理员的二进制文件。
  3. `/usr/lib` 中包含了 `/usr/bin` 和 `/usr/sbin` 用到的库。
  4. `/usr/local` 中包含了从源安装的用户程序。
7. `/lib`：系统库。包含支持位于 `/bin`和`/sbin`下的二进制文件的库文件；库文件名为 `ld*或lib*.so.*`
8. `/proc`：包括系统进行相关信息。这是一个虚拟的文件系统，包含有关正在运行的进程的信息；系统资源以文本信息形式存在。
9. `/var`：系统里的可变数据，变量文件，并不是存放在磁盘上的数据，一般是存放在内存中的数据。
  1. 系统日志文件 `/var/log`
  2. 包和数据库文件 `/var/lib`
  3. 电子邮件 `/var/mail`
  4. 打印队列 `/var/spool`
  5. 锁文件 `/var/lock`
  6. 多次重新启动需要的临时文件 `/var/tmp`



10. `/dev`：包含设备文件，这些包括终端设备、USB或连接到系统的任何设备。例如 `/dev/tty1`
11. `/tmp`：包含系统和用户创建的临时文件，当系统重新启动时，这个目录下的文件都将被删除。
12. `/home`：用home目录来存储他们的个人档案。
13. `/opt`：可选的附加应用程序
14. `/media`：用于挂载可移动设备的临时目录。举例来说，挂载CD-ROM的`/media/cdrom`，挂载软盘驱动器的`/media/floppy`
15. `/srv`：srv代表服务。包含服务器特定服务相关的数据。
16. 修改环境变量PATH，临时修改可以直接 `PATH=$PATH:/bin`，但是要永久生效得修改配置文件 `/etc/profile`

## 文件权限

会考的，编译器不考

1. 文件权限要求掌握，SUID、SGID等权限
  1. sticky：在这个文件夹创建的文件是否具有用户互斥性，比如在一个文件夹中是否允许user1写入的文件被user2删除
  2. SUID：将用户提升到root权限，经典：su
  3. SGID：将组提升到root，经典：sudo
2. 文件权限层次
  1. 用户：创建文件的用户
  2. 组：拥有文件的组中的所有用户
  3. 其他用户：其他
3. 三个类型
  1. 读 r 读取文件或列表内容目录
  2. 写 w 更改文件或创建/删除文件在目录中
  3. 执行 x 以程序执行文件或使用目录作为活动目录

## 进程的概念

写对就行，主观题，判断选择（是否每个进程都有父子进程），daemons不考

进程：是一个正在执行的程序实例，由执行程序、当前值、状态信息以及通过操作系统管理此进程执行情况的资源组成；包含有一个地址空间，在该空间内指定一个或多个线程，并分配这些线程所需的系统资源。

`echo $$` 告知shell当前的进程号

1. 进程的启动都由其他进程启动：所有的进程都是由内核进程(PID=1)来启动的，往往都具有父子进程关系，init(1号进程)由内核本身启动，形成树状的层次结构。
2. 进程的终止
  1. 正常终止
    1. 从main返回
    2. 调用exit函数(库函数，做清理处理)
    3. 调用\_exit函数(系统函数，立即进入内核)
  2. 异常终止

1. 调用abort函数
2. 由一个信号终止

// 例子：打开文件夹

```
DIR *dp;
struct dirent * entry;
if((dp = opendir(dir)) == NULL)
    err_sys();
while((entry = readdir(dp)) != NULL){
    lstat(entry->d_name, &statbuf);
    if(S_ISDIR(statbuf.st_mode)){
    }
    else{}
}
closedir(dp);
```

## 帮助文档

细节不考，知道man和info是就行

1. man命令
  1. 您可以阅读命令的手册页
  2. 手册页存储在/usr/man中
2. info 命令
  1. 用于阅读文档的程序，有时可以代替手册页
  2. 有关信息的信息存储在/ usr / info中

知道图上基本的就行。（库函数在系统调用上面，系统调用不能用库，内核态不能用库）

安卓层次图不考

## 文件命令

locate dir vdir mkfifo chgrp egrep可以不掌握

1. 文件操作
  1. 列出目录内容: ls, dir, vdir **mknod:创建块或字符设备文件；mkfifo：创建命名管道文件**
  2. 创建特殊文件: mkdir, mknod, mkfifo
  3. 文件操作: cp, mv, rm
  4. 修改文件属性: chmod, chown, chgrp, touch
  5. 查找文件: locate, find **find:递归搜索文件系统**
  6. 字符串匹配: grep(egrep)
  7. 其它: pwd, cd, ar, file, tar, more, less, head, tail, cat
2. 进程操作: ps, kill, jobs, fg, bg, nice
3. 其它
  1. who, whoami, passwd, su, uname, ...
  2. man

# 重定向考试重点！

完全掌握！

## 1. 标准输入、标准输出、标准错误

1. 对应的文件描述符：0、1、2
2. C语言变量：stdin、stdout、stderr

## 2. 符号

1. `<`：输入重定向，改变命令的输入，后面指定输入内容(文件名)
2. `>`：输出重定向，将前面输出的部分输入到后面的文件，会删除文件原有的内容。
3. `>!`：同上，强制覆盖。
4. `<<`：追加输入重定向：后面跟字符串，用来表示输入结束
5. `>>`：追加输出重定向：把前面输出的东西追加到后面的文件尾部，不删除文件的内容
6. `2>`：错误重定向：将错误的信息输入到后面的文件中，会删除文件原有的内容
7. `2>>`：错误追加重定向：将错误的信息追加到后面的文件中，不会删除文件原有的内容

8. 例：kill -HUP 1234 `>` killout.txt `2>` killerr.txt

9. 例：kill -HUP 1234 `>` killout.txt `2>&1` 将 kill 命令的标准输出（stdout）和标准错误（stderr）合并重定向到同一个文件。

## 3. 重定向操作在底层本质上就是调用了 `dup2` 函数，将进程的文件描述符进行拷贝和覆盖

## 4. 一般情况下，每个Unix/Linux命令运行时都会打开三个文件：

1. 标准输入文件(stdin)：stdin的文件描述符为0，Unix程序默认从stdin读取数据。
2. 标准输出文件(stdout)：stdout的文件描述符为1，Unix程序默认向stdout输出数据。
3. 标准错误文件(stderr)：stderr的文件描述符为2，Unix程序会向stderr流中写入错误信息。

## 5. 如果希望将stdout和stderr合并后重定向到file，可以这样写：command `>` file `2>&1`或command `>>` file `2>&1`，为什么需要**将标准错误重定向到标准输出**的原因，那就归结为标准错误没有**缓冲区**，而stdout有。

## 6. 如果希望对stdin和stdout都重定向，可以这样写：command `<` file1 `>` file2；command命令**将stdin重定向到file1**，将stdout重定向到file2。

## 7. cat重定向或直接加参数在执行流程上有什么区别？

```
cat < abc.txt
```

```
cat 文件名
```

```
# 这里的文件名作为cat参数存在
```

```
# 类似于C语言中，main(int argc, char** argv)
```

```
# 当argc等于0，在system.in中读，再system.out打印
```

```
# 当cat后面加上参数，就读出文件内容，输出在屏幕上
```

```
# cat < 文件名，这里是重定向
```

```
# 执行流程完全不一样，仍然是走else分支
```

# 管道

理解，输入输出，让大家辨析？

## 1. 将一个进程的输出作为另一个进程的输入

2. 上一个操作的标注输出指的是上一个命令在一系列重定向操作之后的仍然会输出到stdout的文件描述符

3. 例：

1. `ls | wc -l`: 存在一个管道文件, 这个文件作为输入传递给后面的命令
2. `ls -lF | grep ^d`
3. `ar t /usr/lib/libc.a | grep printf | pr -4 -t`

## 环境变量

1. 变量包括 用户变量、环境变量、参数变量和内部变量
2. 操作环境的参数
3. 查看和设置环境变量
  1. `echo $file_parameters`
  2. `env` 查看所有环境变量
  3. `set` 设置环境变量
4. 例: PATH环境变量, 对变量进行读操作添加 `$`, 而写操作不必
  1. `echo $PATH`
  2. `export PATH`: 设置和显示环境变量
5. 具体变量
  1. `HOME` 用户登录目录
  2. `PATH` 问号分割的用来搜索命令的目录清单
  3. `PS1` 命令行提示符, 通常是\$字符
  4. `PS2` 辅助提示符, 用来提示后续输入, 通常是>字符
  5. `IFS` 输入区分隔符
  6. `UID` 当前用户的识别字, 取值由数位构成的字符串
  7. `PWD` 当前工作目录的绝对路径名
6. 参数变量和内部变量(未提)
  1. `$#` 传递到脚本程序的参数个数
  2. `$0` 脚本程序的名字
  3. `$1, 2...` 脚本程序的参数
  4. `$*` 全体参数组成的清单, 使用IFS中的第一个字符分割
  5. `$@` `$*` 的变体, 不使用IFS中的第一个字符分割

## 高级命令与正则表达式

find sed grep等不会考那么难

## shell

### shell进程有什么特点, 是什么程序

Shell是用户与操作系统的接口, 作为核外程序存在。具有双重角色:

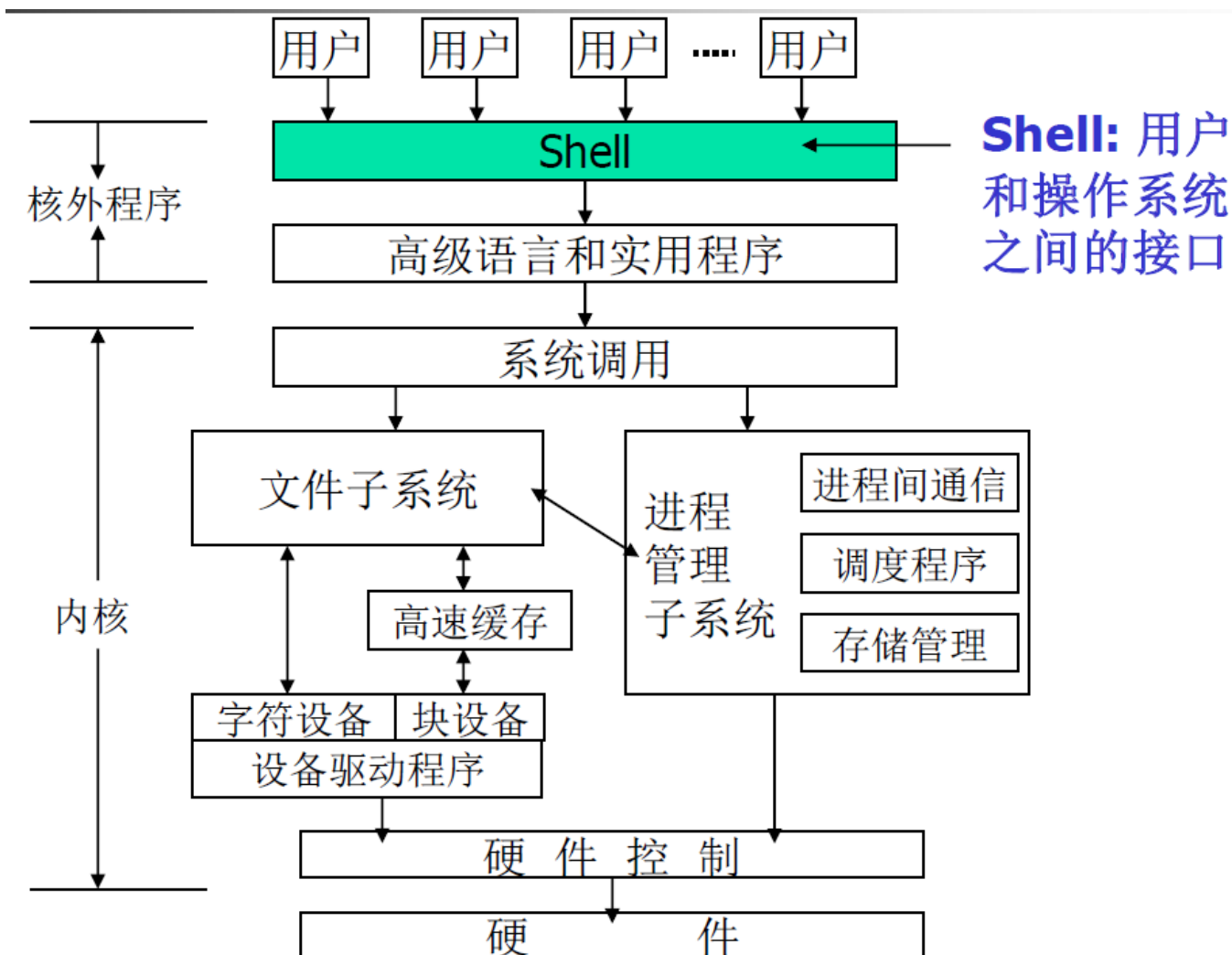
1. Shell是一种命令解释程序:

1. Linux的开发启动过程(进程树)

2. Shell的工作步骤：打印提示符；得到命令行；解析命令；查找文件；准备参数；执行命令。

2. Shell还是一种独立的程序设计语言解释器。

## shell在层次的位置



## shell只考bash，但应该知道其他

1. 不同类型的Shell

| shell名称    | 描述                                     | 位置              |
|------------|----------------------------------------|-----------------|
| ash        | 一个小的shell                              | /bin/ash        |
| ash.static | 一个不依靠软件库的ash版本                         | /bin/ash.static |
| bsh        | ash的一个符号链接                             | /bin/bsh        |
| bash       | “Bourne Again Shell”。Linux中的主角，来自GNU项目 | /bin/bash       |
| sh         | bash的一个符号链接                            | /bin/sh         |
| csh        | C shell, tcsh的一个符号链接                   | /bin/csh        |
| tcsh       | 和csh兼容的shell                           | /bin/tcsh       |
| ksh        | Korn Shell                             | /bin/ksh        |

shell扮演的角色，哪些功能是shell执行的？管道？ls？重定向？哪些是shell执行哪些不是？

## 执行shell的三个方法及其区别

Shell脚本的执行方式

1. 方法1: `sh script_file`, Shell新建进程运行本方法
2. 方法2: `chmod +x script_file` 授予文件可执行权限; `./script_file`, Shell新建进程运行
3. 方法3: `source script_file`; `.script_file`, Shell使用本进程运行

## shell别名，输出环境变量

## 默认执行脚本

## 即时文档要考

1. 在shell脚本中向一条命令传送输入数据
2. Example: `cat >> file.txt << !CATINPUT!`

一些例子

```
read x
case $x in
  1|2) echo "hello";;
  3|4) echo "null";;
  *) echo "yes";;
esac
exit 0
```

```
read x
if [ $x -eq 1 ]
then
  echo "hello"
elif [ $x -eq 2 ]
then
  echo "yes"
else
  echo "null"
fi
exit 0
```

```
read x
while [ $x -le 10 ]
do
  echo $x
  x=$((x+1))
done
exit 0
```

## 编译

### linux可执行码是elf格式

### 编译链接图

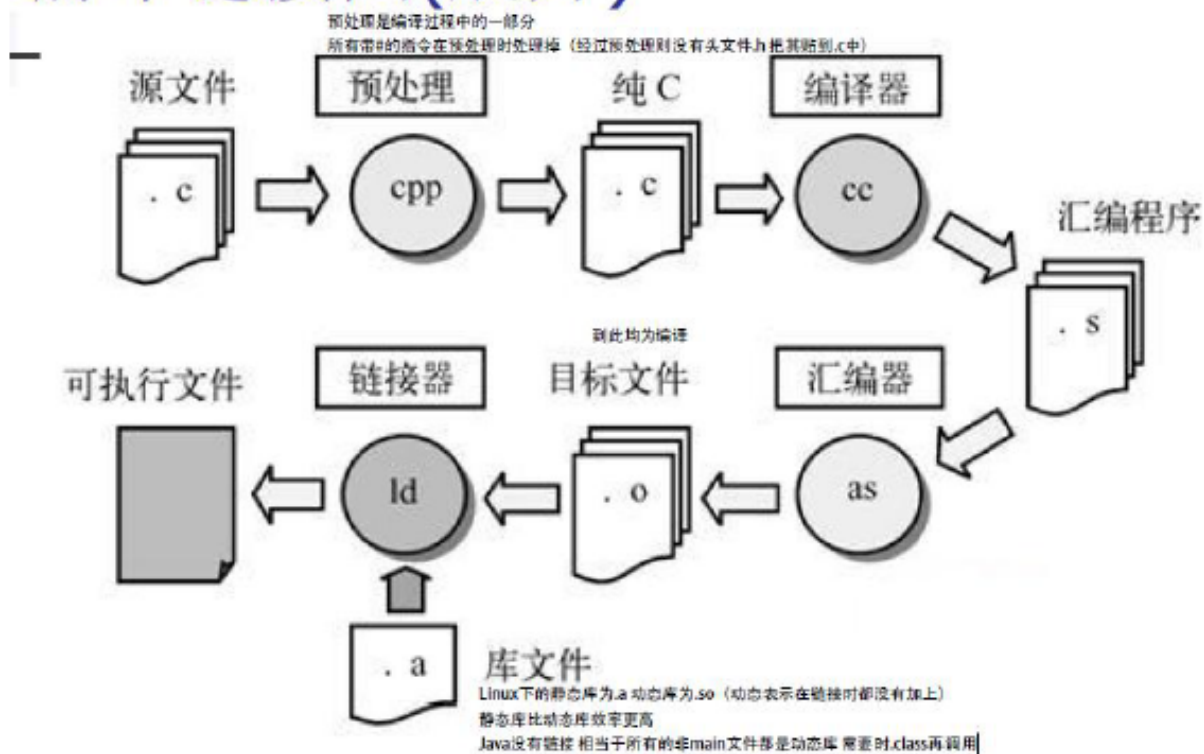
知道大体是是编译和连接两步，做了什么事要知道。

两个.c编译之后有什么？

gcc和g++默认就是进行编译和链接的工作，如果要分开使用就编译：gcc -c

# 编译链接图(展开)

编译过程是一对一，链接过程往往是多对一



1. 每一个源代码和目标文件是一一对应的
2. 链接器是将所有的目标文件进行连接
3. 打包后得到了.a静态库文件

## 静态库和动态库区别

1. 二者的不同点在于代码被载入的时刻不同。
  1. 静态库的代码在编译过程中已经被载入可执行程序，因此体积比较大。
  2. 动态库(共享库)的代码在可执行程序运行时才载入内存，在编译过程中仅简单的引用，因此代码体积比较小。
  3. 不同的应用程序如果调用相同的库，那么在内存中只需要有一份该动态库(共享库)的实例。
  4. 静态库和动态库的最大区别：静态情况下，把库直接加载到程序中，而动态库链接的时候，它只是保留接口，将动态库与程序代码独立，这样就可以提高代码的可复用度，和降低程序的耦合度。
2. 静态库：编译链接时，把库文件的代码全部加入可执行文件中，因此生成的文件比较大，但是运行时也就不需要库文件了，后缀名一般为.a
  1. 为什么需要静态库：通过静态库的方式降低复杂度，在升级更新时尽量做到增量更新，但是静态库会导致复用性降低，磁盘占用高。
3. 动态库：在编译链接时并没有把库文件的代码加入可执行文件中，而是在程序执行时由运行时链接文件加载库，这样可以节省系统的开销，后缀名一般为.so。
  1. 动态库的作用
    1. 库文件不在可执行文件中，放置在外侧
    2. 升级更新会方便快捷



3. 动态库会存在冲突(版本问题)

4. gcc/g++在编译时默认使用动态库。无论静态库还是动态库，都是由.o文件构成的

## Delphi不考，java&.Net要能说出来

### 编译器结构

知道前端终端后端

## GCC

命令掌握、参数也要掌握，扩展名知道.a .so .o .h .c即可

1. Usage:

1. gcc [options] [filename]

2. gcc 链接(如果文件为.o，则自动只做链接，生成可执行文件) 或 编译+链接

1. `-E`：只对源程序进行预处理(调用cpp预处理器)

2. `-S`：只对源程序进行预处理、编译

3. `-c`：执行预处理、编译、汇编而不连接

4. `-o`：output\_file：输出执行文件名

5. `-g`：产生调试工具所必须的符号信息

6. `-O/On`：在程序编译、连接过程中进行优化处理

7. `-Wall`：显示所有的警告信息

8. `-ldir`：指定额外的头文件搜索路径，dir是文件路径

9. `-Ldir`：指定额外的库文件搜索路径，dir是文件路径

10. `-lname`：链接时搜索指定的库文件

11. `-DMACRO[=DEFN]`：定义MACRO宏，比如 `gcc -DAA=2` 相当于源码中添加了 `#define AA 2`

3. g++ C++对应命令

`gcc -o hello hello.c`：将hello.c文件编译成hello的可执行文件

`gcc -c hello.c`：将hello.c文件生成hello.o文件

`gcc -E hello.c`：只是激活预处理，不生成文档，需要把它重定向到另外一个文档里

`gcc -S hello.c`：将hello.c文件生成hello.s文件的汇编代码

`gcc -pipe -o hello hello.c`：使用管道代替编译中临时文档

`gcc hello.c -include /root/hello.h`：包含某个代码，简单来说，就是以某个文档，需要另外一个文档的时候，就能够用它设定，功能就相当于在代码中使

## 调试器(GDB)

调试器原理需要掌握，用法不考

### 调试器原理

1. `file` 打开要调试的文件

2. `break/tbreak` 设置调试

3. `run` 执行当前调试的程序

4. `list` 列出源代码一部分

5. `next` 执行一步但不进入函数
6. `step` 执行一步进入函数
7. `display` 显示表达式的值
8. `print` 临时显示表达式的值
9. `kill` 中止调试
10. `quit` 退出

## MakeFile

简单（展开来写的）的要求掌握，缩写形式不会就算了但要求能读。可以不会写，但要会读。函数不要求掌握，不用看

1. Makefile描述模块间的依赖关系，会记录所有文件的信息，在make时决定链接时需要重新编译哪些
2. 如果找到，它会找文件中的第一个目标文件(target)
3. 如果文件不存在，或后依赖的.o文件的修改时间新于这个文件，那么则会执行后面所定义的命令来生成这个文件。

## make

1. 命令根据Makefile对程序进行管理和维护
2. make命令格式: `make [-f Makefile][option][target]`
3. make判断被维护文件的时序关系
4. make install 需要root权限

## makefile语法

1. 缩进
  1. 顶格的为规则
  2. 缩进的为命令
2. makefile语法
  1. target是一个目标文件
  2. prerequisites是生成target所需的文件或目标
  3. command是make需要执行的命令
  4. 默认回显：如果不想要回显则在命令前添加@
  5. "%.o"表明要所有以".o"结尾的目标，即"foo.o bar.o"，就是变量\$object集合的模式

```
target: prerequisites
    command
```

3. 举例：依赖关系，如果后面的文件更新，则执行如下代码，若输出文件不存在是执行如下代码则为违反规则。

```
hello : main.o kbd.o
    gcc -o hello main.o kbd.o
main.o : main.c defs.h
    cc -c main.c
kbd.o : kbd.c defs.h command.h
    cc -c kbd.c
clean :
    rm edit main.o kbd.o # 伪目标
```

1. make会在当前目录下找名字叫"Makefile"或"makefile"的文件。
2. 查找文件中的第一个目标文件(target), 举例中的hello
3. 如果hello文件不存在, 或是hello所依赖的文件修改时间要比hello新, 就会执行后面所定义的命令来生成hello文件。
4. 如果hello所依赖的.o文件不存在, 那么make会在当前文件中找目标为.o文件的依赖性, 如果找到则再根据那个规则生成.o文件。(类似一个堆栈的过程)
5. make根据.o文件的规则生成 .o 文件, 然后再用 .o 文件生成hello文件。

## 伪目标

```
clean:
    rm *.o hello
```

1. 伪目标并不是一个文件, 只是一个标签, 所以make无法生成器依赖关系和决定它是否要执行, 只能显式的指明目标才能使其生效。
2. 伪目标取名不能和文件名重名: 可以使用 `.PHONY` 标记来显示地指明一个目标是伪目标, 向make说明, 不管是否有这个文件, 这个目标是伪目标
3. 伪目标一般无依赖文件, 但是也可以执行
4. 伪目标可以作为默认目标, 只要放到第一个

## makefile例子

```
TOPDIR = ../
include $(TOPDIR)Rules.mak
EXTRA_LIBS += :
EXEC = $(INSTALL_DIR)/hello
# make时会找到makefile文件, 找到其中的第一个目标文件, 如果不存在或旧的, 则会执行后面命令来生成hello文件
OBJS = hello.o # make uninstall之后系统中源代码仍然存在
# 变量定义, makefile可以include别的makefile

all: $(EXEC) # 默认执行make all
    $(EXEC): $(OBJS)
    $(CC) $(LDFLAGS) -o $@ $(OBJS) $(EXTRA_LIBS) # gcc的别名CC, $@明确了目标文件放置位置
install:
    $(EXP_INSTALL) $(EXEC) $(INSTALL_DIR) # make install执行的指定目标
clean:
    -rm -f $(EXEC) *.elf*.gdb *.o
```

## 预定义变量

1. `$<` 第一个依赖文件的名称
2. `$?` 所有的依赖文件，以空格分开，这些依赖文件的修改日期比目标的创建日期晚
3. `$+` 所有的依赖文件，以空格分开，并以出现的先后为序，可能包含重复的依赖文件
4. `$^` 所有的依赖文件，以空格分开，不包含重复的依赖文件
5. `$*` 不包括扩展名的目标文件名称
6. `$@` 目标的完整名称
7. `$%` 如果目标是归档成员，则该变量表示目标的归档成员名称

## 系统调用

### 文件、文件系统概念，考主观题

1. 文件：是数据的集合，可以写入、读取或两者兼有的对象(文件具有某些属性，包括访问权限和类型)。逻辑上是字节，文件必然是整数字节。
2. 文件结构：字节流(Linux)、记录序列、记录树
3. 文件系统：操作系统中负责访问和管理文件的部分，是文件及其某种属性的集合，为引用文件的文件序列号提供了名称空间。
  1. 一种特定的文件格式
  2. 指按指定格式进行格式化的一块存储介质
  3. 指操作系统中(通常内核中)用来管理文件系统以及对文件进行操作的机制及其实现

### 文件类型！

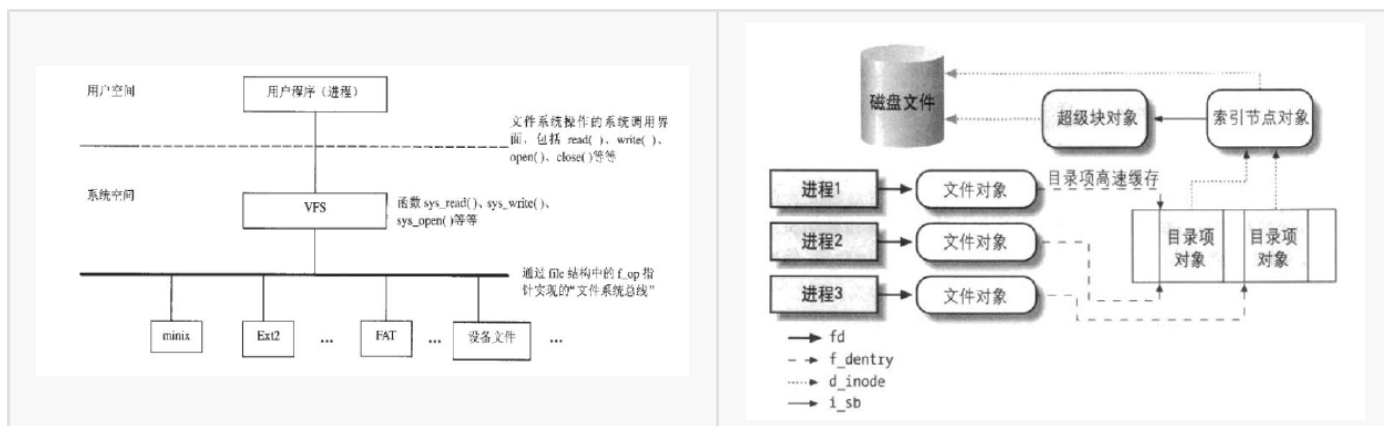
1. `-` 普通文件：文件或代码数据，没有特别的内部结构
2. `d` 目录
3. `l` 符号链接
4. `s` 套接字文件(网络接口):启动一个程序来监听客户端的要求，而客户端可以通过这个socket来进行数据的沟通
5. `b` 块设备文件：发大量数据，比如移动硬盘，字符设备按块读取
6. `c` 字符设备文件：发少量数据，按字符读取
7. `p` 命名管道文件：结果多个程序同时存取一个文件的错误问题

## VFS重点

VFS模型、原理、有哪几个对象

### 4.3.1. 什么是VFS

1. 采用标准的Unix系统调用读写位于不同物理介质上的不同文件系统，使得open()等系统调用不用关心底层的存储介质和文件类型
2. VFS: Virtual File System Switch



#### VFS在系统内建立的四中结构体和含义

1. 超级块(super block): 某一个磁盘的某一个分区的文件系统的信息，记录了文件系统类型和参数。
2. i-node 对象(i-node object): 记录真正的文件，文件存储在磁盘上时是按照索引号访问文件的，软链接是不同的文件，但是硬链接是相同的inode号，同一个文件。
3. 文件对象(file object): 记录了文件描述符、索引号，不对应真正的文件，文件打开会创建出文件对象，文件关闭才会释放内核中的文件对象。记录了文件的读写状态。
4. 目录对象(dentry object): 维护了目录中的逻辑关系，若要通过目录来查找文件，都需要这个对象。在路径上，无论是目录还是文件，都是一个dentry对象对应到目录包含的i-node上，目录项包括索引节点编号，目录项名称长度以及名称。

## 文件系统在磁盘上的格式完全不考

## 硬链接和软链接的区别！

1. 硬链接
  1. 不同的文件名对应同一个inode
  2. 不能跨越文件系统
  3. 对应系统调用link
  4. ln [原文件名] [连接文件名]
2. 软链接
  1. 存储被链接文件的文件名(而不是inode)实现链接
  2. 可以跨越文件系统
  3. 对应系统调用symlink
  4. ln -s [原文件名] [连接文件名]

## 系统调用和库函数有什么联系和区别

1. 都以C库函数的形式出现
2. 系统调用: 需要切换到内核态来进行相应的操作，编译运行速度和效率高，是Linux内核对外的位移接口(用户

程序和内核之间的唯一接口), 提供出**最小接口**。

3. 库函数: 库函数的可移植性好, 依赖于系统调用, 提供较为**复杂的功能**, 例如标准I/O库

## 考试编程

---

使用与文件系统相关的系统调用 or C库来实现功能, 注意考试要求, 不能用混

- 一般要求使用系统调用
- 函数参数小区别不扣分, 离谱的会扣分
- 以系统调用为主, 库函数也要看
- 系统调用与库函数同样功能上的区别

## 内核

---

考概念为主

### 内核概念

---

很容易考到!!!

1. 操作系统是**一系列程序的集合**, 其中**最重要的部分**构成了内核
2. 单内核/微内核
  1. 单内核是一个很大的进程, 内部可以分为若干模块, 运行时是一个独立的二进制文件, 模块间通讯通过直接调用函数实现
  2. 微内核中大部分内核作为**独立的进程**在特权下运行, 通过消息传递进行通讯
3. Linux内核的**能力**: 内存管理, 文件系统, 进程管理, 多线程支持, 抢占式, 多处理支持
4. Linux内核区别于其他UNIX商业内核的优点
  1. 单内核, 模块支持
  2. 免费/开源
  3. 支持多种CPU, 硬件支持能力非常强大
  4. Linux开发者都是非常出色的程序员.
  5. 通过学习Linux内核的源码可以了解现代操作系统的实现原理

### 内核层次结构知道就行

---

### 源码获取方式、编译步骤不考

---

**启动内核的配置、引导要掌握 (kernel initrd initrd), kernel initrd initrd怎么获得不考**

---

### 打包不考

---

### 驱动命令都要

---

- 许多常见驱动的源代码集成在内核源码里
- 也有第三方开发的驱动, 可以单独编译成模块.ko

- 编译需要内核头文件的支持

#### 1. 加载驱动

- `insmod driver.ko` 强制加载命令，`insmod`命令需要到`.ko`当前文件夹下执行
- `modprobe driver.ko` 前提是要创建文件夹 `/lib/modules/$(uname -r)/`，然后把驱动模块`.ko`放到以上文件夹里面，再执行`depmod`命令。然后`modprobe`命令就可以使用了，并且该命令可以在任意文件夹下执行

#### 2. 卸载驱动

- `rmmod driver` 强制卸载命令，同上。注意：卸载没有`.ko`
- `modprobe -r driver` 同上

#### 3. 列出已安装驱动模块

- `lsmod` 列出后，可以观察一下基本信息，有的时候可能会显示`pernament`（永久驻留），即无法卸载。这个命令和`cat /proc/modules`等价

#### 4. 显示某个驱动信息

`modinfo driver.ko` //用法同 `modprob`

#### 5. 显示驱动主设备号

`cat /proc/devices`

#### 6. 创建设备节点： `mkmod /dev/device_name c MAJOR MINOR`

`c`代表字符设备，也可以是`b`（代表块设备），`MAJOR`主设备号，`MINOR`次设备号

linux一切皆文件，设备也是文件，安装驱动后一定要创建设备节点（也就是文件），以便应用程序通过设备节点（文件）操作底层驱动

- `lsmod` 列出已经加载的内核模块；
- `modinfo` 查看模块信息；
- `modprobe` 挂载新模块以及新模块相依赖的模块
- `rmmod` 移除已挂载模块；
- `depmod` 创建模块依赖关系的列表
- `insmod` 挂载模块；

## 模块间依赖知道是怎么回事

---

1. 一个模块**A**引用另一个模块**B**所导出的符号，我们就说**模块B被模块A引用**。
2. 如果要装载模块**A**，必须先要装载模块**B**。否则，模块**B**所导出的那些符号的引用就不可能被链接到模块**A**中。这种模块间的相互关系就叫做**模块依赖**。
3. 模块是为了完成某种特定任务而设计的。其功能比较的单一，为了丰富系统的功能，所以模块之间常常进行通信。其之间可以共享变量，数据结构，也可以调用对方提供的功能函数。

## 内核代码、用户态代码的联系和区别

---

| 用户态程序     | 内核态程序              |
|-----------|--------------------|
| 用户空间运行    | 内核空间运行             |
| 入口是main() | 入口由module_init()指定 |
| 没有出口      | 出口由module_exit()指定 |
| 直接运行      | 程序由insmod命令载入      |
| gdb调试     | kdbug、kdb、kgdb等调试  |

## 写内核的注意事项

- 1. 不可以使用C库来开发驱动程序
- 2. 没有内存保护机制
- 3. 小内核栈
- 4. 并发上的考虑

## helloworld内核代码尽量不考，考概念为主

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/init.h>
static int __init hello_init(void)
{
    printk(KERN_INFO "Hello world\n");
    return 0;
}
static void __exit hello_exit(void)
{
    printk(KERN_INFO "Goodbye world\n");
}
module_init(hello_init);
module_exit(hello_exit);
```

- 1. static int \_\_init hello\_init(void)
- 2. static void \_\_exit hello\_exit(void)
  - 1. Static声明，因为这种函数在特定文件之外没有其它意义
  - 2. \_\_init标记, 该函数只在初始化期间使用。模块装载后，将该函数占用的内存空间释放
  - 3. \_\_exit标记 该代码仅用于模块卸载。
- 3. Init/exit
  - 1. 宏： module\_init/module\_exit
  - 2. 声明模块初始化及清除函数所在的位置
  - 3. 装载和卸载模块时， 内核可以自动找到相应的函数
    - 1. module\_init(hello\_init);



2. module\_exit(hello\_exit);

## 怎么编译内核模块不考

---

## 跟硬件打交道不考

---

## 传参不考

---

## 内核间的依赖，知道导出的符号有哪几种，查看导出符号的命令要掌握

---

1. 如果一个模块需要向其他模块导出符号(方法或全局变量)，需要使用:
  1. EXPORT\_SYMBOL(name);
  2. EXPORT\_SYMBOL\_GPL(name);
2. 注意:符号必须在模块文件的全局部分导出，不能在函数部分导出。更多信息可参考 <linux/module.h>文件
3. Modules仅可以使用由Kernel或者其他Modules导出的符号不能使用Libc
4. `cat /proc/kallsyms` 可以显示所有导出的符号

## proc文件是个什么东西，主观题，相关函数不考

---

# 内核模块操作/proc文件

- **/proc文件系统**，这是**内核模块和系统交互**的两种主要方式之一。
- /proc文件系统也是Linux操作系统的特色之一。
- /proc文件系统不是普通意义上的文件系统，它是一个**伪文件系统**。
- 通过/proc，可以用标准Unix系统调用(比如**open()**、**read()**、**write()**、**ioctl()**等等)**访问进程地址空间**
- 可以用**cat**、**more**等命令查看/proc文件中的信息。
- 用户和应用程序可以通过/proc得到系统的信息，并可以改变内核的某些参数。
- 当调试程序或者试图获取指定进程状态的时候，/proc文件系统将是**你强有力的支持者**。通过它可以创建更强大的工具，获取更多信息。

# 驱动有哪几种类型

---

狭义上，驱动程序专指操作系统中的用来操控硬件的代码

分类：

- 字符设备驱动。软件操作设备时，以字节为单位进行的。典型的如LCD、串口、LED、蜂鸣器、触摸屏。
- 块设备驱动。相对于字符设备定义的。软件操作设备时，以块(多个字节构成的一个单位)为单位的。设备的块大小是设备本身设计时定义好的，软件不能去更改，但不同设备的块大小可以不一样。常见的块设备都是存储类设备，如硬盘、NandFlash、iNand、SD。
- 网络设备驱动。网络设备是专为网卡设计的驱动模型。linux中网络设备驱动主要目的是为了支持API中socket相关函数的工作。

分类原则是设备读写操作的特征差异；

## 建立字符型设备的具体步骤，主观题

---

1. 申请设备号：每一个字符设备或块设备都有一个主设备号或次设备号
  1. 主设备号：表示一个特定的驱动程序
  2. 次设备号：表示使用该驱动程序的各设备
2. 定义文件操作结构体file\_operations
3. 创建并初始化定义结构体cdev
  1. cdev结构体描述字符设备
  2. 该结构体是所有字符设备的抽象，其包含了大量字符设备所共有的特性。
4. 将cdev注册到系统，并和对应的设备号绑定
5. 在/dev文件系统中用mknod创建设备文件，并将该文件绑定到设备号上
  1. 设定设备号: `device=scull`
  2. 定义主设备号: `major=15`
  3. 用户可以通过访问 `/dev/scull10` 来访问当前的驱动设备

4. `mknod /dev/${device}0 c $major 0`

## 整个接口原理、驱动建立之后用户态程序如何使用

---

### 设备号

---

知道设备号是什么东西。具体函数不考，读都不读！

## 华为讲课内容

---

1. 2019年底EulerOS被正式推送开源社区，命名为openEuler
  1. openEuler是一个开源、免费的Linux发行平台
  2. 支持x86、ARM、RISC-V等多种处理器架构

3. 所有开发者、企业、商业组织都可以使用openEuler社区版本，也可以基于社区版本发布自己二次开发的操作系统版本。
2. openEuler：毕昇JDK
  1. **ARM64优化**：dmb指令消除等提升性能
  2. **快速序列化技术**：提升序列化，反序列化性能
  3. **GC优化**：让系统减少卡顿
  4. **SpecJBB** 提升20%
3. 线程间通信ITC
  1. 互斥机制：openEuler提供了"NUMA感知队列自旋锁"实现互斥机制，减小了NUMA体系结构中使用自旋锁的开销。
  2. 同步机制：openEuler中提供down原语不up原语，能够实现线程的同步运行。
4. 进程间通信IPC：openEuler增强了两种进程间通信机制，共享内存与消息传递机制
5. OpenEuler内存页说明
  1. 各级页表的表项大小为8B
  2. OpenEuler将标准大页封装成一个**伪文件系统**提供给用户程序申请并访问。
  3. OpenEuler采用LRU最近最久未使用策略实现页选择唤出。
  4. 页在未来被访问的概率只能预测，不能精准判断。
6. 鲲鹏处理器
  1. 鲲鹏处理器是基于ARMv8-64位RISC指令集开发的通用处理器
  2. 使用大量寄存器：通用X0-X30(31个，64位) + 特殊寄存器 + 系统寄存器
7. openEuler的增强
  1. openEuler对通用Linux操作系统作了增强
  2. 提供iSulad轻量级容器全场景解决方案
8. 为了充分发挥鲲鹏处理器的优势，openEuler在多核调用技术、软硬件协同、轻量级虚拟化、指令集优化和智能优化引擎等方面做了增强。
9. openEuler提供"NUMA感知队列自旋锁"实现互斥机制以减小MUMA体系结构中使用自旋锁的开销。
10. openEuler通过提供KAE(鲲鹏加速引擎，Kunpeng Accelerator Engine)插件，使能KunPeng硬件加速能力，包括
  1. 对称/非对称加密
  2. 数字签名
  3. 压缩解压缩等算法，用于加速SSL/TLS应用和数据压缩